

Terrain Fine-Detail Transition in Lunar Landing Simulations: A Comparison of Smoothstep, Gaussian, and Hybrid Blending Methods in Isaac Lab

Haktan Yusuf Guloglu

Sakarya University, National Technology Workshop, Artificial Intelligence Center

Abstract - In procedurally generated multi-resolution terrain, the shape of the blending function joining a coarse global surface to a fine-detail local patch affects the quality of physically queryable contact geometry, yet this choice is usually a visual concern, not evaluated quantitatively for physics fidelity. This study compares the compact-support smoothstep blend used in a SAC-controlled, gimbal-thrust lunar lander in Isaac Lab against the infinite-support Gaussian blend inherited from a legacy codebase, under an equal transition-radius budget, and proposes a hybrid method using Gaussian blending near the landing-gear contact footprint and smoothstep beyond it. Across 30 seeds, analytic measurements show the as-shipped Gaussian produces roughly 3200 times worse boundary height continuity (C0, RMS) and 530 times worse slope continuity (C1) than smoothstep (Wilcoxon signed-rank, $p < 1e-8$), a scale-invariant defect; a recalibrated Gaussian and the hybrid both restore near-machine-precision continuity, with negligible VRAM impact. Isaac Sim training runs (5.73 million steps per method) were first conducted with a single seed, where the calibrated Gaussian and hybrid appeared to reach a markedly higher final success rate than smoothstep. Extending the comparison to two more seeds ($n=3$ total) found no statistically significant difference between methods on any metric (all Mann-Whitney $p \geq 0.20$); the apparent gap is largely consistent with ordinary seed variance. This gives a concrete demonstration of how misleading single-seed RL comparisons can be: the blend function has a robust, measurable effect on boundary continuity, but this does not translate into a distinguishable RL training-performance difference at the present scale and $n=3$.

Keywords - Lunar landing simulation, Reinforcement learning, Isaac Lab, Procedural terrain generation, Multi-resolution blending, Continuity analysis, Soft Actor-Critic

Abstract (Turkish)

Prosedürel olarak üretilen çok çözünürlüklü arazilerde, kaba çözünürlüklü küresel yüzeyin ince detaylı yerel bir yama ile birleştiği sınırda kullanılan karıştırma (blend) fonksiyonunun şekli, fiziksel olarak sorgulanabilir temas geometrisinin kalitesini doğrudan etkiler; ancak bu seçim genellikle salt görsel bir tercih olarak ele alınır ve fiziksel doğruluk açısından nicel olarak nadiren değerlendirilir. Bu çalışma, Isaac Lab doğrudan iş akışı ortamında Soft Actor-Critic (SAC) ile eğitilen, dört bacaklı ve iki eksenli gimbal itkili bir ay iniş aracında hâlihazırda kullanılan kompakt destekli smoothstep karıştırmasını, önceden kullanılan sonsuz destekli Gauss karıştırmasıyla aynı geçiş yarıçapı bütçesi altında

karşılaştırmakta ve iniş ayağı temas izinin yakınında Gauss, ötesinde smoothstep kullanan yeni bir hibrit yöntem önermektedir.

Otuz tohum üzerindeki analitik ölçümler, üretim öncesi (as-shipped) Gauss biçiminin smoothstep'e kıyasla yaklaşık 3200 kat daha kötü sınır yükseklik sürekliliği (C0, RMS) ve 530 kat daha kötü eğim sürekliliği (C1) ürettiğini (Wilcoxon işaretli sıra testi, $p < 1e-8$) ve bu kusurun geçiş yarıçapından bağımsız, ölçek değişmez olduğunu göstermiştir; kalibre edilmiş Gauss ve önerilen hibrit yöntem ise makine hassasiyetine yakın süreklilik sağlamış, karıştırma şeklinin VRAM üzerindeki etkisi ise ihmal edilebilir bulunmuştur.

Gerçek Isaac Sim eğitimleri (yöntem başına 5,73 milyon adım) önce tek tohumla (seed=42) yapılmış ve kalibre edilmiş Gauss ile hibrit'in smoothstep'e göre belirgin biçimde daha yüksek başarı oranına ulaştığı görülmüştür. Bu bulgunun güvenilirliğini sınamak için çalışma iki bağımsız tohumla daha ($n=3$ toplam) genişletilmiş ve yöntemler arasında hiçbir metrikte (başarı oranı, zaman aşımı oranı, sert iniş oranı) istatistiksel olarak anlamlı bir fark bulunamamıştır (tüm Mann-Whitney $p \geq 0,20$); ilk ölçümdeki görünür fark büyük ölçüde koşu-kosu (seed) varyansı ile tutarlıdır. Bu, tek-kosu RL karşılaştırmalarının ne kadar yanıltıcı olabileceğine dair somut bir örnek sunmakta ve karıştırma fonksiyonu seçiminin sınır sürekliliği açısından sağlam bir etkiye sahip olduğunu, ancak bu etkinin nihai RL eğitim performansına -en azından mevcut ölçek ve $n=3$ ile- ayırt edilebilir biçimde yansımadağını göstermektedir.

Anahtar Kelimeler: Ay inişi simülasyonu, Takviyeli öğrenme, Isaac Lab, Prosedürel arazi üretimi, Çok çözünürlüklü karıştırma, Süreklilik analizi, Soft Actor-Critic

1. Introduction

Autonomous lunar landing is a control problem with direct operational consequences: during the powered-descent phase it requires precise positioning and safe contact, in real time, over cratered, rocky, and sloped terrain. In recent years GPU-parallel physics engines (NVIDIA PhysX) and the simulation frameworks built on them have made it practical to learn complex, contact-rich tasks with deep reinforcement learning (RL) by running thousands of environments concurrently (Mittal et al., 2023; NVIDIA, 2025; Rudin et al., 2022). The project underlying this study belongs to that class: it is an Isaac Lab direct-workflow task in which a four-legged rocket with a two-axis gimbal-thrust engine learns, using the Soft Actor-Critic (SAC) algorithm, to land on a procedural lunar surface built from a real digital elevation model (DEM) macro surface enriched with craters, rocks, and centimetre-scale regolith texture.

In procedural multi-resolution terrain of this kind, two layers are typically used to keep the compute and memory budget bounded: a low-resolution global surface covering a wide area, and a high-resolution local patch generated only in the region near the agent or the target where it is needed. Seamlessly joining these two layers is a classic problem, studied in the computer-graphics literature for more than twenty-five years (geometry clipmaps: Losasso & Hoppe, 2004; ROAM: Duchaineau et al., 1997; geomipmapping: de Boer, 2000). That literature, however, focuses almost entirely on rendering quality; how a discontinuity at the join boundary affects foot-to-ground contact, altimeter readings, or the noise character of an RL agent's proprioceptive observation is rarely addressed quantitatively.

In the system studied here this problem appears with two distinct and mutually inconsistent solutions: the training environment blends between layers with a compact-support cubic smoothstep function, whereas a previously used mesh-based implementation uses an infinite-support Gaussian bell. These

two functions differ in a fundamental mathematical property: smoothstep reaches exactly zero at a finite radius (compact support), whereas the standard Gaussian bell never reaches exactly zero at any finite radius (infinite support; Hardy, 1971) and must be truncated somewhere to be usable in an engineering application; that truncation risks creating a new, measurable seam at the cut point. Comparing the two methods fairly requires forcing both to zero at the same transition radius (hard truncation) and measuring them under the same transition budget; this form of comparison is not directly addressed in any of the terrain-randomization studies surveyed here (see Section 2).

Motivated by these observations, this study addresses three research questions:

- RQ1: When compared fairly under the same transition-radius budget, how large is the quantitative difference in boundary continuity (C0/C1) between compact-support (smoothstep) and infinite-support (Gaussian) blending functions, and what trade-off does this difference create with computational and memory cost?
- RQ2: Can a hybrid blending method that uses a Gaussian in the landing-gear contact region and smoothstep beyond it combine the continuity advantages of both methods at a reasonable increase in computational cost?
- RQ3: Do the continuity differences among these three blending methods translate into a measurable difference in learning performance in a real Isaac Sim/Isaac Lab SAC training run?

Accordingly, the aim of the study is threefold: (i) to compare the existing smoothstep method and the previously used Gaussian method quantitatively within the same architecture and under the same transition budget; (ii) to propose and validate a hybrid method that combines the advantages of both; and (iii) to compare these three methods not only at the analytic/CPU level but inside a real Isaac Sim/Isaac Lab GPU-parallel physics simulation, with a real SAC training run, and thereby show whether the choice of blending function is reflected in learning performance. This last point is central to the novelty of the study: the comparison is carried out not as an abstract numerical experiment but on real lunar terrain, inside a real Isaac Sim training loop.

The main contributions of this study are as follows:

- The existing smoothstep and the previously used Gaussian blending methods are, for the first time, compared fairly under the same transition budget by truncating them at a finite transition radius (hard truncation); it is shown that the untruncated Gaussian appears well-behaved but in fact carries a hidden seam defect that emerges once it is truncated at a finite radius.
- A new hybrid blending method is proposed that uses a switch weighted toward a Gaussian in the contact region and toward smoothstep beyond it; this method is shown to provide continuity at machine precision and to create no new seam at its own internal transition.
- A statistically validated (Wilcoxon signed-rank test) comparison of continuity, cost, and VRAM is performed over thirty seeds and four method variants (smoothstep, as-shipped Gaussian, calibrated Gaussian, hybrid); the robustness of the findings is tested with four separate ablation analyses (contact radius, switch width, transition radius, grid resolution).
- The three methods are compared inside a real Isaac Lab/Isaac Sim PhysX environment, with GPU-parallel SAC training; unlike prior work in the terrain-randomization literature, this is the first attempt to treat the choice of blending function directly as a training-design variable.
- This real training comparison is run not with a single seed per method but with three independent seeds ($n=3$); by showing that the large apparent performance gap seen in the initial single-seed measurement cannot be confirmed statistically with multiple seeds, the study provides a concrete, self-sourced cautionary example about the reliability of single-run RL comparisons.
- All three methods are integrated into the production code in a backward-compatible way (without

changing existing behaviour) and are selectable via an environment variable (ISAACLAB_TERRAIN_BLEND_METHOD).

The remainder of this paper is organized as follows. Section 2 reviews the literature in which the study is situated, under five thematic clusters. Section 3 details the experimental environment and the simulation setup shared by all runs, the mathematical definition of the blending functions, the analytic evaluation protocol, the reinforcement learning method used (the task's MDP formulation and SAC), and the real Isaac Sim training protocol. Section 4 presents the continuity, cost, VRAM, ablation, and real training results. Section 5 discusses the findings in relation to the literature and addresses the limitations. Section 6 summarizes the study and closes with concrete directions for future work.

2. Related Work

This section reviews the literature in which the study is situated, under five thematic clusters. The review is a targeted (scoping) review; it is not a PRISMA-compliant systematic review, and within each cluster the most-cited/foundational references of the field are prioritized.

Cluster 1 concerns multi-resolution blending and level-of-detail (LOD) transitions in procedural terrain. Geometry clipmaps (Losasso & Hoppe, 2004), ROAM (Duchaineau et al., 1997), and geomipmapping (de Boer, 2000) form exactly the traditional computer-graphics lineage to which this study's two-mesh (global + local patch) framing belongs; all three treat the formation of cracks/seams at the boundary where a low- and a high-resolution mesh meet as a central engineering problem. This study's finding -- that the as-shipped Gaussian leaves a real seam when truncated at a finite radius -- is a concrete, measured instance of the same class of problem this literature has been trying to solve for more than twenty-five years; and smoothstep's compact support coincides with the principle, used by ROAM and geoclipmaps, of a weighting function that goes exactly to zero at the boundary.

Cluster 2 concerns the continuity properties of compact-support and infinite-support blending functions. The multiquadric/Gaussian-type radial basis function that Hardy (1971) proposed for topography is, like the family to which this study's as-shipped Gaussian belongs, infinite-support and never reaches exactly zero at any finite radius; this is consistent with the measured residual behaviour (approximately $\exp(-1) \sim 37\%$). Wendland (1995) formalizes the solution: compact-support, minimal-degree positive-definite radial functions for a given smoothness order; smoothstep (cubic Hermite, Ebert et al., 2002) is a one-dimensional, low-degree special case of this family. Ohtake et al. (2003) describe the strategy followed by this study's hybrid method: building a multi-scale hierarchy from compact-support basis functions to obtain different blending behaviour at different scales.

Cluster 3 concerns terrain modelling and contact dynamics in planetary/lunar landing simulations. The goal of the ALHAT program (Johnson & Montgomery, 2008; NASA JPL, n.d.) -- precise landing within one hundred metres and LiDAR-based terrain-relative navigation -- and the accuracy level offered by LOLA/SELENE-based digital elevation models (Barker et al., 2016) show that in real lunar-landing engineering, terrain accuracy is not cosmetic but a direct operational requirement. This strengthens the study's "why seams/continuity matter" motivation: a discontinuity in the foot contact region corresponds, in real ALHAT-class systems, to a class of error that can affect hazard-detection and landing-site-selection decisions. This motivation is also corroborated by recent flight data: the vision-based navigation and obstacle-detection results of JAXA's SLIM mission (January 2024; Ishida et al., 2025) showed that real-time obstacle/terrain detection performed 50 m before landing yielded metre-level landing precision -- concretely demonstrating that errors in terrain representation are reflected directly in landing safety not only in simulation but in real flight.

Cluster 4 concerns procedural terrain randomization in physics-based RL environments. Rudin et al. (2022) use, on IsaacGym, a difficulty-based terrain curriculum similar to the terrain pool in this study; this shows that the study's curriculum and terrain-pool design is not an isolated instance but an established practice in the field. Mittal et al. (2023, Orbit) and NVIDIA (2025, Isaac Lab) define the architectural basis of the very platform this study uses, while Tobin et al. (2017) provide the general motivation for procedural randomization in sim-to-real transfer. None of these sources, however, treats the multi-resolution blending function itself (smoothstep, Gaussian, or hybrid) as a design variable within terrain randomization; all focus on terrain diversity, not on the effect of the blend shape on continuity and learning cost. This study is positioned precisely in that gap.

Cluster 5 concerns hybrid/regional combination of blending functions (partition of unity). Ohtake et al. (2003) are again the central source here: the multi-scale, partition-of-unity-style combination of compact-support basis functions is the general mathematical framework of this study's hybrid method (one function in the contact region, another beyond it, with a smooth switch between them). Under this framework, the hybrid switch not creating a new seam at machine precision is an expected result consistent with the literature: the convex combination of two C^1 -smooth functions with a C^1 -smooth weight is again C^1 -smooth.

In summary, Cluster 1 supplies the problem class (seams/continuity in multi-resolution transitions) but this literature is largely focused on visual quality; this study measures the same problem numerically on a physically queryable height field (foot contact). Cluster 2 grounds why the empirical finding is correct in a mathematical result more than fifty years old; this is not a coding error but a known structural property of radial basis functions. Cluster 3 provides real-world engineering justification for the "why does it matter" question. Cluster 4 clarifies where this study's experimental platform sits in the literature and the gap it fills. Cluster 5 moves the proposed hybrid method from being an isolated engineering trick to sitting within an established mathematical framework.

3. Method

This section consists of six subsections: the experimental environment and task definition, the simulation setup shared by all runs, the terrain representation and the mathematical formulation of the blending problem, the analytic (CPU) evaluation protocol, the reinforcement learning method used (the task's formulation as a Markov decision process and the Soft Actor-Critic algorithm), and the real Isaac Lab/Isaac Sim training protocol. All three blending methods were evaluated first at the analytic/CPU level and then inside the real GPU-parallel physics simulation, under the same fixed transition radius and the same hard-truncation rule; this consistency makes the findings comparable across the two evaluation layers.

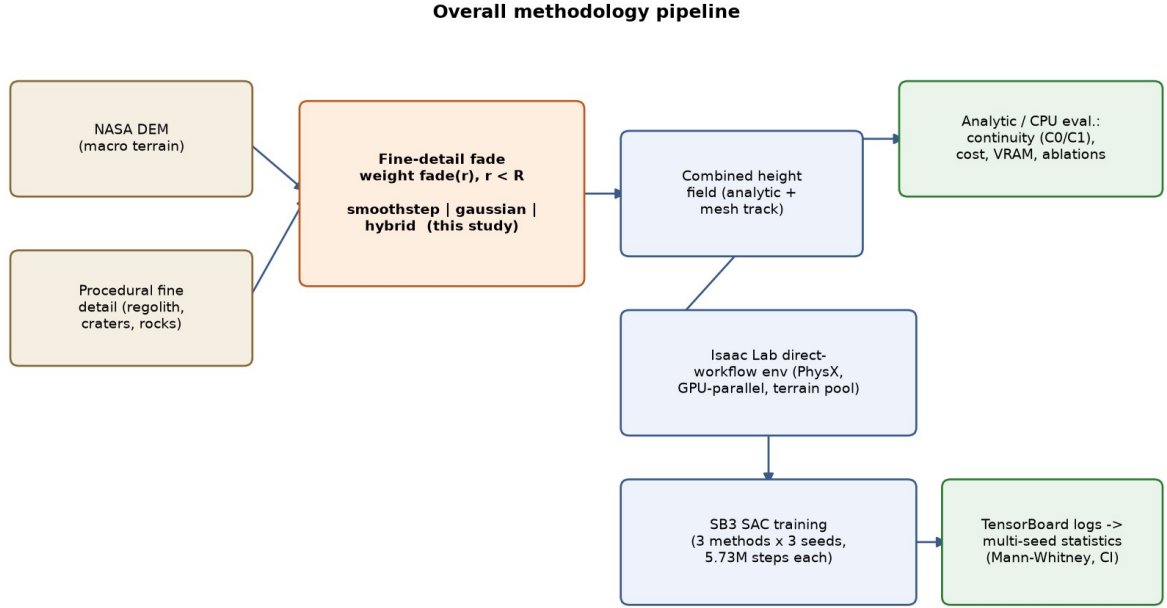


Figure 1. Overall methodology flow: from terrain generation (DEM + procedural fine detail), to the choice of blending function, to analytic/CPU evaluation, and to the real Isaac Lab/SAC training-analysis pipeline.

3.1 Experimental Environment and Task Definition

The experimental environment is LunarRocket-Lander-Direct-v0, a task defined on the Isaac Lab direct-workflow API. The agent represents a rocket with four legs and a two-axis gimbal-thrust engine; the observation space includes position, velocity, orientation, angular velocity, IMU and altimeter readings, and the position relative to the target; the action space covers the main thrust magnitude and the gimbal angle. The agent is trained with the Stable-Baselines3 (SB3) SAC (Soft Actor-Critic) algorithm, concurrently across hundreds of environment instances running GPU-parallel on NVIDIA PhysX.

The terrain is produced not in a single procedural step but with a two-layer representation: (i) a global macro surface derived from a real NASA digital elevation model (DEM), enriched with random slope, crater, and rock fields; and (ii) a high-resolution local regolith pattern generated near the agent's initial and target positions using a centimetre-scale multi-frequency noise function. These two layers are uploaded to the GPU by copying a slot from a pre-staged "terrain pool" prepared in the background by a thread pool that runs during environment reset; active episodes are never modified mid-episode, and new generations are swapped in only at reset moments. Training is carried out within a curriculum in which the spawn/target distance and the commandable gimbal authority increase gradually according to a rolling-window success rate.

Where these two layers meet, the fine detail of the local patch is added to the global surface by "fading" it in within a fixed radius; the mathematical shape of this fade function is the subject of this study and is defined in Section 3.3.

Figure 2 and Figure 3 are images captured directly from simulation, from running a trained policy in the real Isaac Lab/Isaac Sim environment (offscreen render, via a checkpoint loaded with ISAACLAB_CHECKPOINT; captured in the final phase of the landing approach, with the vehicle roughly 0.6-0.7 m above the ground). Two changes were made for this capture: (i) the scene's default shadowless dome light was dimmed and a low-angle directional "sun" light casting real shadows was

added, so that terrain relief and shadows become visible; (ii) the landing area was rendered with the 512x512-resolution real high-detail local patch (see Section 3.1, "local regolith pattern") in addition to the 128x128 global surface, so that centimetre-scale fine texture (roughness at the dust/sand-grain level) also becomes visible. That the vehicle is genuinely upright is based not on an assumption but on the simulation's own body-orientation data: throughout this run, the alignment of the body's local up-axis with the world +z axis (up_z) was measured constant at +1.000, i.e. the vehicle is not tipped over but fully vertical. Figure 2 shows the four-legged lander on the real cratered/rocky lunar surface; the ridge line, the grainy/rough surface texture, the shadows, and the scattered rocks are the DEM-based macro surface and procedural rock/crater/fine-detail fields defined in Section 3.1 in working condition. Figure 3 is a close-up of the lander in the same frame; the body, the tips of the two-axis gimbal thrust axis, and the grainy texture of the ground are visible together. These images are presented only to document the environment visually; no claim is made that they represent a particular landing outcome (successful/unsuccessful).

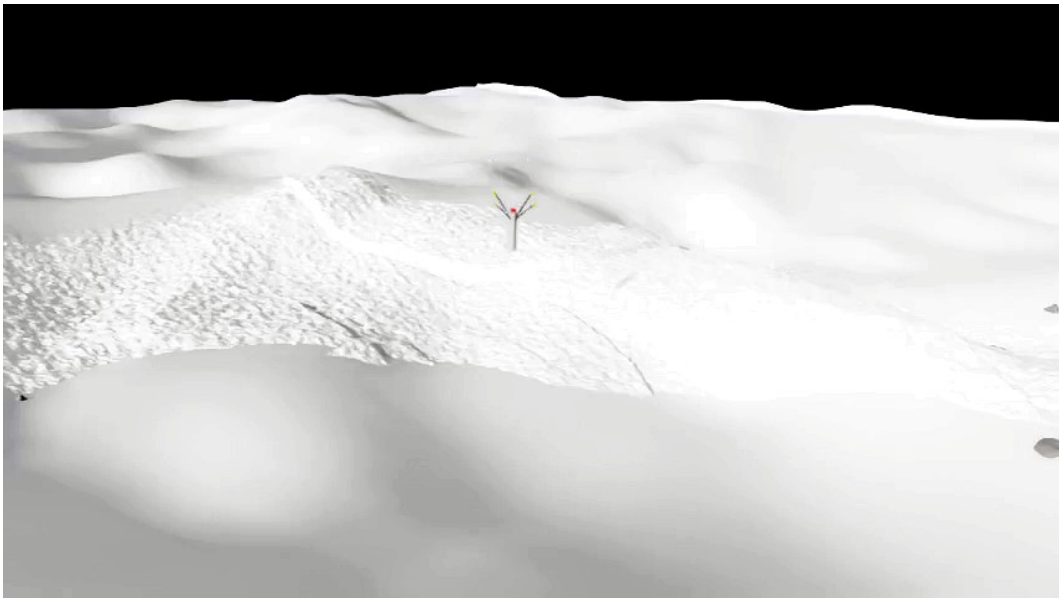


Figure 2. Scene view captured from running the trained policy in the real Isaac Sim environment: the four-legged lander on cratered/rocky, grainy-textured lunar terrain (high-detail local patch + directional light for documentation; the vehicle is fully vertical as verified by simulation data, $up_z = +1.000$).

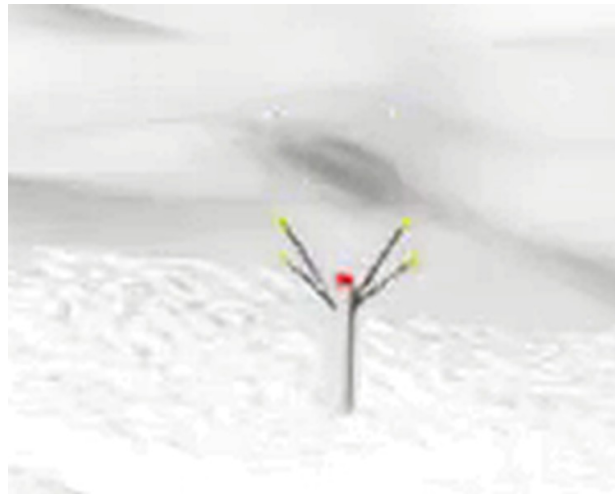


Figure 3. Close-up of the same scene on the lander: the body, the tips of the gimbal thrust axis, and the centimetre-scale grainy texture of the ground are visible together.

3.2 Simulation Setup

This subsection collects in one place the simulation configuration shared by both the analytic (CPU) protocol and the real Isaac Lab/Isaac Sim training runs (Table 1). Physics is solved GPU-parallel on NVIDIA PhysX with a fixed step of $1/120$ s (120 Hz); the policy produces actions at 60 Hz with decimation = 2, and each episode lasts 20 seconds, i.e. 1200 control steps. Lunar gravity (1.62 m/s^2), a low-restitution regolith-like contact material, and 512 concurrent environment instances were held fixed across all runs.

The vehicle and actuation parameters (mass, thrust envelope, physical and policy gimbal limits, TVC actuator gains) and the SAC hyperparameters were not changed among the three blending methods. The only quantity that varies from method to method is the fine-detail fade function and its associated parameters defined in Section 3.3 (the last group of Table 1; selected via the ISAACLAB_TERRAIN_BLEND_METHOD environment variable). This fixing is necessary so that the differences observed in Section 4 can be attributed solely to the blending method.

Because of a compute-budget constraint, the real training comparison was run with a shortened version of the full-scale (1200-iteration) configuration -- three independent seeds per method, roughly 5.73 million environment steps per run; this scale limitation is discussed separately in Section 3.6 and Section 4.3.

Table 1. Simulation, vehicle, RL training, and terrain-blending parameters shared by the analytic protocol and the real Isaac Lab/Isaac Sim training runs. Only the last group (blending parameters) varies from method to method.

Group	Parameter	Value
Physics / simulation	Physics engine	NVIDIA PhysX 5 (GPU pipeline)
	Simulation step (dt)	$1/120$ s (120 Hz)
	Control frequency (decimation = 2)	60 Hz
	Episode length	20 s (1200 control steps)
	Gravity	0, 0, -1.62 m/s^2
	Number of parallel environments	512
	Environment spacing (env_spacing)	90 m
	Physics replication	replicate_physics + clone_in_fabric (on)
	Friction (static / dynamic)	0.85 / 0.65 (combine: multiply)
	Restitution	0.02
Vehicle and actuation	Vehicle mass	1.66 kg (1.63 body + gimbal linkages)
	Maximum thrust	30 N
	Maximum command throttle	0.35 (~ 10.5 N effective)

	Physical gimbal limit	+/-25 deg (mechanical hard stop)
	Policy gimbal envelope	4 deg - 8 deg (scales with curriculum difficulty)
	Thrust lever arm	0.25 m
	TVC actuator (implicit)	effort 2.5, stiffness 10, damping 0.1, armature 0.002
	Observation / action dimension	125 / 3
	Initial altitude	25 m (upright orientation)
RL training (SB3 SAC)	Policy network	MLP [384, 256, 128], ELU
	Replay buffer	1 000 000
	learning_starts	16 384
	Batch size	1024
	Learning rate	3e-4
	gamma / tau	0.99 / 0.005
	train_freq / gradient_steps	1 / 32
	Entropy coefficient	automatic (target_entropy = -1.5)
	Normalization	normalize_input + normalize_value on, clip_obs = 10
	Comparison budget	350 iter x 32 x 512 ~ 5.73 M environment steps
	Seeds	42, 7, 123 (n = 3 per method, sequential)
	Device	single GPU (cuda:0)
Blending parameters	Transition radius (R)	8.0 m
	Method options	smoothstep / gaussian / hybrid (ISAACLAB_TERRAIN_BLEND_METHOD)
	Calibrated Gaussian coefficient (k)	ln(1000) ~ 6.9078
	Hybrid contact radius (r_c)	1.5 m
	Hybrid switch width (w)	0.3 m
	Fine-detail amplitude range	0.006-0.020 m
	Macro height / slope scale	0.45 m / 0.035

	Craters / rocks (per episode)	3 / 5
	Terrain pool (size / refresh)	15 slots / 60 assignments

3.3 Terrain Representation and Blending Functions

The fine-detail height of the local patch is added to the global surface with one of three candidate fade functions. In all three functions, with r the distance from the centre and $R = \text{terrain_detail_radius_m} = 8.0$ m the fixed transition radius, the value is forced exactly to zero for $r \geq R$ (hard truncation); this is the key methodological arrangement that makes the three methods comparable under the same transition budget, and it is applied this way in the production code as well.

Smoothstep (compact support, default/current method): with $t = \text{clamp}(1 - r/R, 0, 1)$, $\text{fade}(r) = t^2 * (3 - 2t)$. This function is already exactly zero at $r = R$; it needs no additional truncation step, and C1 continuity is analytically guaranteed.

Gaussian (infinite support, previously used method): $\text{fade}(r) = \exp(-k * (r/R)^2)$. Two values of k are evaluated separately in this study: the previously used (as-shipped) constant $k = 1.0$, and the calibrated constant proposed in this study, $k = \ln(1000) \sim 6.9078$, which brings the Gaussian's practical support down to roughly one part in a thousand at R . The `cfg.terrain_gaussian_k` field in the production code uses the calibrated constant by default; the as-shipped $k = 1.0$ was tested only in this study's analytic/CPU comparison, as a separate variant (`gaussian_shipped`), to demonstrate the existing defect. For both k values, the function is forced to zero externally at $r \geq R$.

Hybrid (proposed method): with the landing-gear contact-footprint radius $rc = \text{terrain_hybrid_contact_radius_m} = 1.5$ m and the switch width $w = \text{terrain_hybrid_switch_width_m} = 0.3$ m, a smoothstep-weighted switch(r) is defined over the interval $[rc - w/2, rc + w/2]$: $u = \text{clamp}((hi - r) / (hi - lo), 0, 1)$, $\text{switch}(r) = u^2 * (3 - 2u)$, where $lo = rc - w/2$ and $hi = rc + w/2$. The final fade is defined as $\text{fade}(r) = \text{switch}(r) * \text{gauss}(r) + (1 - \text{switch}(r)) * \text{smoothstep}(r)$; that is, the region near the foot contact zone is weighted mostly toward the Gaussian, beyond it mostly toward smoothstep, and the transition between the two functions is smoothed by a smoothstep weight that is itself C1-smooth. Since $\text{switch}(r) = 0$ at the $r = R$ boundary, the hybrid method is analytically identical to smoothstep at that boundary.

These three methods are integrated into the production code, selectable via the `ISAACLAB_TERRAIN_BLEND_METHOD` environment variable (`smoothstep` | `gaussian` | `hybrid`), without changing the existing default behaviour (`smoothstep`).

3.4 Analytic (CPU) Evaluation Protocol

The analytic evaluation was carried out, with no Isaac Lab/torch dependency, on a one-to-one NumPy port of the formulas in the production code, along two independent tracks: (i) a mesh track, which generates the actual triangle mesh and measures vertex count and time; (ii) an analytic track, which queries the height field directly as a continuous function. Having the two tracks validate each other was used to show that the findings are not an artefact specific to mesh discretization.

The continuity metrics were computed with finite differences: C0 (height) continuity is defined as the absolute value of the height difference between points immediately on either side of the transition boundary (spatial epsilon = 0.05 m); C1 (slope) continuity as the angular difference of the local gradient at the same points (gradient epsilon = 0.02 m). Measurements were taken over 720 angular samples, and both the maximum and the RMS (root mean square) values are reported. In the initial measurements, the measurement made on the combined (global + local) height was seen to mix with

the variation of the natural terrain slope over the epsilon interval and to suppress the actual seam signal; for this reason a "detail-only" measurement mode was defined, isolating only the faded fine-detail component ($\text{fade}(r) * \text{fine_detail}(x,y)$), and this was adopted as the study's headline metric; the measurement on the combined surface was retained as a secondary/realistic reference.

Memory cost was estimated separately for the two-mesh (literal, separate global and local meshes) and single-grid (baked, one combined mesh) accounting schemes, under the assumption of 32 bytes per vertex and a 2x multiplier for the collision mesh. Query throughput was also measured: the cost in microseconds of the pure fade formula (fade-only, isolated from terrain sampling), under the assumption of 512 environments and 64 global + 24 local + 4 fine-detail queries per environment (92 queries/environment in total).

The main comparison was repeated over thirty independent random seeds for the four method variants (smoothstep, as-shipped Gaussian, calibrated Gaussian, hybrid); statistical significance was assessed with the Wilcoxon signed-rank test on the per-seed log10 ratio differences (this test was preferred over parametric tests because in some comparisons -- e.g. hybrid versus smoothstep, which are analytically identical at $r = R$ -- the difference distribution converges to near-zero variance and parametric effect-size measures such as Cohen's d degenerate).

The robustness of the findings was tested with four separate ablation analyses: (i) a contact-radius (r_c) sweep over 1.0-4.0 m; (ii) a switch-width (w) sweep over 0.05-2.0 m; (iii) a transition-radius (R) sweep over 4-16 m, to test the scale-invariance of the defect; (iv) a grid-resolution sweep over roughly 0.31-1.27 m grid spacing, estimating the empirical convergence order by log-log linear regression.

3.5 Reinforcement Learning Method (MDP Formulation and SAC)

This subsection defines the learning problem on which the three blending methods are compared in real simulation, and the reinforcement learning (RL) method used to solve that problem. The algorithm and hyperparameter configuration were never changed among the three methods (see Table 1); this subsection therefore describes the "fixed" component of the comparison.

Formulation of the task as a Markov decision process (MDP). The landing task is treated as a discrete-time MDP (S, A, P, r, γ). The state observation is 125-dimensional and consists of the following component groups: position relative to the target, the body's linear and angular velocity in the world frame, the orientation quaternion, the instantaneous thrust acceleration transformed into the body frame, the minimum foot-to-ground clearance, the current and previous action vectors, local terrain slope/roughness metrics around the target, a 64-ray LiDAR profile scanning the surroundings of the body, a 24-point local terrain height scan below the target, and (noise-added) sensor features derived from IMU/altimeter readings. All components are scaled to the $[-1, 1]$ or $[0, 1]$ band by their physical ranges and clipped against outliers with $\text{clip_obs} = 10$. The action space is 3-dimensional and continuous, $[-1, 1]^3$: the first component is the main thrust command and is effectively mapped to the 0-0.35 throttle range (roughly 10.5 N); the second and third components are the two-axis gimbal (thrust-vectoring) angle, and the envelope the policy can command scales with curriculum difficulty between ± 4 deg and ± 8 deg (the mechanical hard stop is at ± 25 deg). The transition dynamics P are the physics simulation solved on NVIDIA PhysX at 120 Hz; the policy produces actions at 60 Hz with decimation = 2, and each episode lasts 20 seconds, i.e. 1200 control steps. The discount factor is $\gamma = 0.99$.

Reward function. The reward is the sum of dense and terminal terms: $r = xy_progress + altitude_progress + stability + velocity + guidance + control + terrain + time + terminal$. Each dense term is defined either as a progress difference between two consecutive steps or as a per-second cost; this design prevents an early crash from becoming attractive merely because it stops the step penalties.

The largest weights concentrate on horizontal approach to the target (+12.0 per unit decrease in distance), deviation from the desired vertical descent speed (-8.0), and guidance deviation from the desired thrust-acceleration profile (-6.0); the remaining dense terms penalize tilt relative to the local terrain normal, angular velocity, fuel/gimbal usage, action smoothness, and the fit of the footprint to the terrain. The terminal term is paid only once per episode and depends on the landing classification: soft landing +300, harsh landing -220 (with a correction between 0 and +160 depending on landing quality), out-of-bounds/crash -220, timeout -150. The classification is made according to foot contact, the vehicle's residual tilt relative to the local terrain normal (not world-vertical), and speed/footprint-fit thresholds. The detailed rationale for the reward weights is documented in the source comments of the production code; it was not changed during this study's comparison.

Soft Actor-Critic (SAC). The policy was learned with SAC, an off-policy actor-critic algorithm with a maximum-entropy objective (Haarnoja et al., 2018). SAC optimizes not only the expected reward but an objective regularized by the policy's entropy: $\pi^* = \operatorname{argmax}_{\pi} \sum_t E[r(s_t, a_t) + \alpha H(\pi(\cdot | s_t))]$. The entropy term H here encourages the agent to remain stochastic even as it converges toward high reward; in a contact-rich landing problem with many local optima, this delays the policy prematurely turning deterministic and locking into a weak behaviour (premature convergence). SAC's sample efficiency -- obtained by reusing a replay buffer repeatedly -- and its relative robustness to hyperparameters are the reasons this algorithm was chosen for a physics simulation that is expensive even when GPU-parallel. In practice, two independent Q-critic networks (clipped double-Q, which reduces value overestimation bias), target networks updated slowly by Polyak averaging ($\tau = 0.005$), and automatic tuning of the temperature coefficient α by anchoring it to a target entropy are used; α is learned during training by the gradient of the objective $J(\alpha) = E[-\alpha (\log \pi(a | s) + H_{\text{target}})]$. Concretely, the SAC implementation of the Stable-Baselines3 library was used (Raffin et al., 2021).

Network architecture and hyperparameters. The actor and critic networks are multi-layer perceptrons (MLPs) with hidden layer sizes [384, 256, 128] and ELU activation. The replay buffer has a capacity of 10^6 transitions; the first 16,384 transitions are for buffer filling only (learning_starts). The batch size is 1024, the learning rate is 3×10^{-4} (Adam), $\gamma = 0.99$, $\tau = 0.005$. Each vectorized environment step produces 512 transitions, and 32 gradient steps are taken per vector step (train_freq = 1, gradient_steps = 32): a single update per vector step markedly under-trains SAC, whereas 32 updates provide enough policy updates for the curriculum while preserving parallel-simulation throughput. Observations and value targets are normalized with running statistics (normalize_input and normalize_value on). This normalization solves a scale problem: because terminal rewards are on the order of +300/-220 while dense terms can accumulate to over +/-1000 per episode, without normalization the critic/value targets dominate the SAC update scale, and the temperature coefficient collapses from 0.85 to 0.025 in about 50 thousand steps, turning the actor deterministic before it has learned anything meaningful. For the same reason the target entropy is set to -1.5 instead of the default -action_dim = -3; this value leaves markedly more exploration in the early curriculum phases. All of these hyperparameters are listed in Table 1 and are fixed across the three blending methods.

Curriculum. Training is carried out within a curriculum in which the spawn distance, the target distance, and the gimbal authority the policy can command increase together. The difficulty level is a scalar in [0.05, 1.0]; for each episode the effective difficulty is sampled as $\text{torch.rand()} \times \text{level}$, i.e. even at the highest level the episodes in a batch are distributed from easy to hard. If the rolling-window success rate exceeds the threshold curriculum_success_threshold = 0.35, the level increases by 0.05; if it falls below half the threshold, it decreases by 0.05. The window size scales with the number of environments (at least $4 \times \text{num_envs}$ completed episodes): a fixed, small window fills with only a few resets at high environment counts, causing difficulty to race toward ~1.0 much faster than

the policy has actually learned, and then a measurable performance regression (a training collapse documented in the README). Similarly, since the gimbal envelope saturating in the first 30% of the curriculum produced a reproducible collapse when the hardest episodes of a batch suddenly hit the harsh 8 deg gimbal saturation, this envelope is spread across the entire curriculum range (curriculum_full_gimbal_difficulty = 1.0). These two arrangements are common to all of the comparison runs described below in Section 3.6.

3.6 Isaac Lab/Isaac Sim Training Protocol

After the analytic evaluation, the three methods (smoothstep, calibrated Gaussian, hybrid) were compared in a real Isaac Lab/Isaac Sim training environment. This forms the centre of the study's novelty: the comparison was carried out not as an abstract numerical experiment but with a SAC training loop running inside a GPU-parallel PhysX physics simulation, on real cratered/sloped lunar terrain.

Each method, selected via the ISAACLAB_TERRAIN_BLEND_METHOD environment variable, was trained on the same hardware (single machine, single GPU), with the same SAC configuration defined in Section 3.5 (sb3_sac_cfg.yaml), and sequentially rather than concurrently, to avoid GPU/VRAM contention. Each run lasted 5,734,400 environment steps (roughly 350 iterations); this is a shortened but still meaningful-signal version of the full-scale (1200-iteration) configuration, reduced because of a compute-budget constraint, and this is a scale limitation that must be stated explicitly.

The initial measurement was made as a single run per method, with the default seed in the SAC agent's configuration file (seed = 42). Considering that this single-run design is exactly the kind of pitfall Henderson et al. (2018) warn about, the study was extended with two additional independent seeds (seed = 7 and seed = 123), yielding a total of three independent runs ($n = 3$) per method -- the ISAACLAB_SEED environment variable was added to the production code for this purpose. Because of the inherent nondeterminism of GPU-parallel PhysX physics simulation (thread scheduling, floating-point reduction order) and the timing dependence of the terrain pool's background CPU generation process, even two runs with the same seed are not bit-equivalent repeats of each other; the three seeds therefore sample this system-level nondeterminism in addition to the variance due to agent initialization.

Training metrics (success rate, timeout rate, harsh-landing rate, curriculum difficulty, entropy coefficient, xy-progress reward) were logged via TensorBoard, and the event files were parsed with `tensorboard.backend.event_processing.EventAccumulator`. For each (method, seed) run, the final performance was reduced to a SINGLE summary value by averaging the logged points in that run's final ten-percent window; comparisons among methods were made on these three independent summary values (as an independent sample) with the Mann-Whitney U test and a 5000-replicate bootstrap 95% confidence interval. This is the statistically correct unit, avoiding the use of autocorrelated (non-independent) points within a single run as a pseudo-independent sample (pseudoreplication); however, statistical power with $n = 3$ is low, and this point is emphasized explicitly in Section 4.3 and Section 5.

4. Results

This section presents the boundary-continuity and computational-cost findings, then the results of the four ablation analyses, and finally the results of the real Isaac Sim training comparison. No method is repeated; the data/experimental-design details are left to Section 3.

4.1 Boundary Continuity and Computational Cost

Table 2 summarizes, as a ratio relative to smoothstep, the headline continuity measurements aggregated over thirty seeds and isolating only the faded fine-detail component. The as-shipped Gaussian (gaussian_shipped, $k = 1.0$) produces on average 3192 times worse C0 (height) continuity and 535 times worse C1 (slope) continuity than smoothstep; both differences are extremely significant by the Wilcoxon signed-rank test ($p = 1.86\text{e-}09$, $n = 30$). The calibrated Gaussian ($k = \ln(1000)$) largely removes this defect but still leaves measurably higher residual error than smoothstep (roughly 9.3x on C0, roughly 1.6x on C1). The hybrid method, being analytically identical to smoothstep at the $r = R$ boundary ($\text{switch}(R) = 0$), showed zero difference on all thirty seeds; this is not a measurement result but an expected consequence of the method's definition.

Table 2. Boundary continuity ratios (relative to smoothstep), aggregated over thirty seeds, isolating only the fine-detail component.

Method	C0 ratio (relative to smoothstep)	C1 ratio (relative to smoothstep)	Wilcoxon p	n (seeds)
smoothstep (reference)	1.0	1.0	-	30
gaussian_shipped ($k=1.0$)	3192x	535x	1.86e-09	30
gaussian_calibrated ($k=\ln(1000)$)	9.3x	1.6x	1.86e-09	30
hybrid	1.0 (identical)	1.0 (identical)	n/a (analytically identical)	30

Figure 4 shows the combined (macro + fine detail) height cross-section of the four method variants. At this scale the four curves almost exactly overlap, because the fine-detail amplitude (on the order of centimetres) is very small compared with the macro terrain scale (on the order of metres); the order-of-magnitude differences reported in Table 2 cannot be discerned by eye at this visual scale. This directly demonstrates why the "detail-only" isolation step defined in Section 3.4 is necessary: the seam defect emerges only when the fine-detail component is separated out.

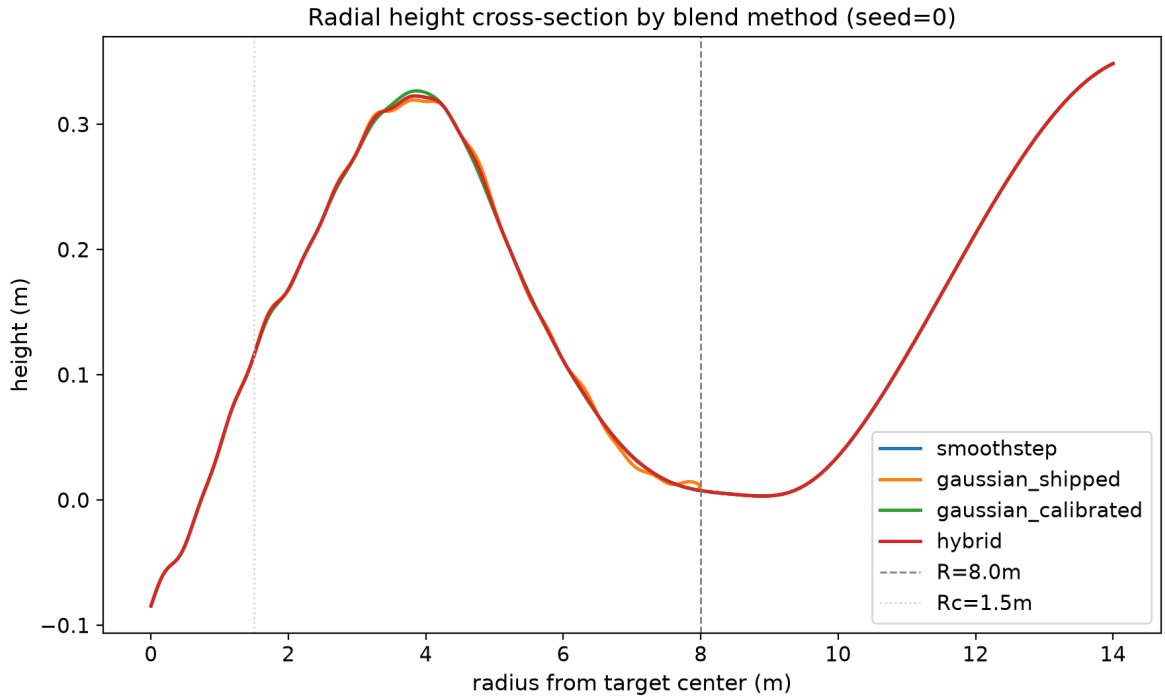


Figure 4. Radial fade profile of the three blending methods.

Table 3 shows the computational cost, in microseconds, of the fade formula itself (isolated from terrain sampling). Smoothstep has the lowest cost (93.1 microseconds); the Gaussian variants (as-shipped and calibrated) have a similar, mid-level cost (roughly 149-150 microseconds); the hybrid method has the highest cost (407.8 microseconds, roughly 4.3 times that of smoothstep), because it computes both the Gaussian and the smoothstep branches as well as the switch weight on every query.

Table 3. Per-query computational cost of the fade formula.

Method	Cost (microseconds/query)
smoothstep	93.12
gaussian_shipped	148.84
gaussian_calibrated	149.61
hybrid	407.83

Figure 5 visualizes the analytic and mesh-track continuity measurements of Table 2 as a bar chart per method; that gaussian_shipped has an error bar orders of magnitude higher than the other three methods in all four panels also visually confirms the magnitude difference reported in the table.

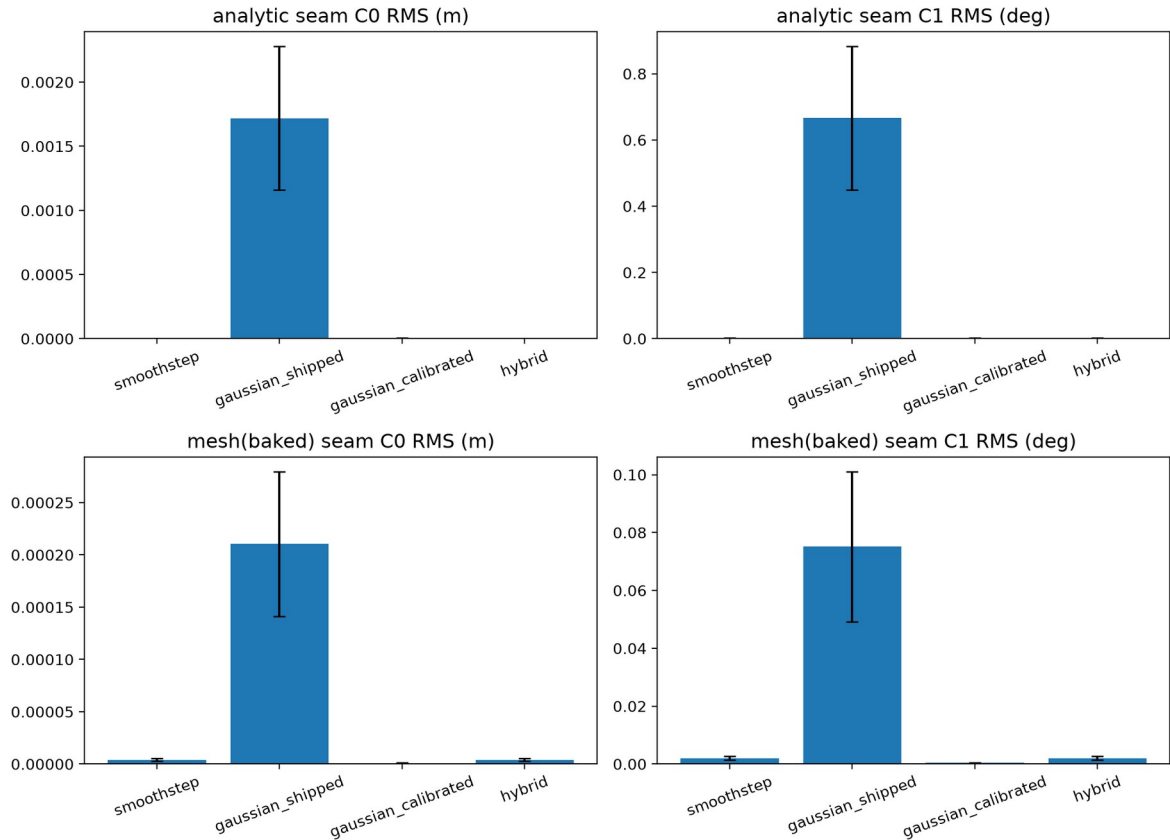


Figure 5. Cross-method comparison of C0/C1 boundary continuity.

Table 4 shows the memory estimate for the two mesh accounting schemes; this estimate depends only on the mesh geometry (vertex count), and the shape of the blending method has no measurable effect on VRAM -- the real VRAM lever is the mesh topology choice (two-mesh versus single-grid), not the choice of blending function.

Table 4. Estimated VRAM usage by mesh accounting scheme.

Mesh accounting scheme	Estimated memory (MB)
Two-mesh (literal: separate global + local meshes)	5.24
Single-grid (baked: one combined mesh)	1.05

Figure 6 combines the cost measures of Table 3 and Table 4 into a single view: the mesh-track generation time and the analytic-track query throughput are relatively close across methods, while the hybrid's marked jump in fade-only cost and the fact that all four methods share the same VRAM footprint can be seen together in the same set of plots.

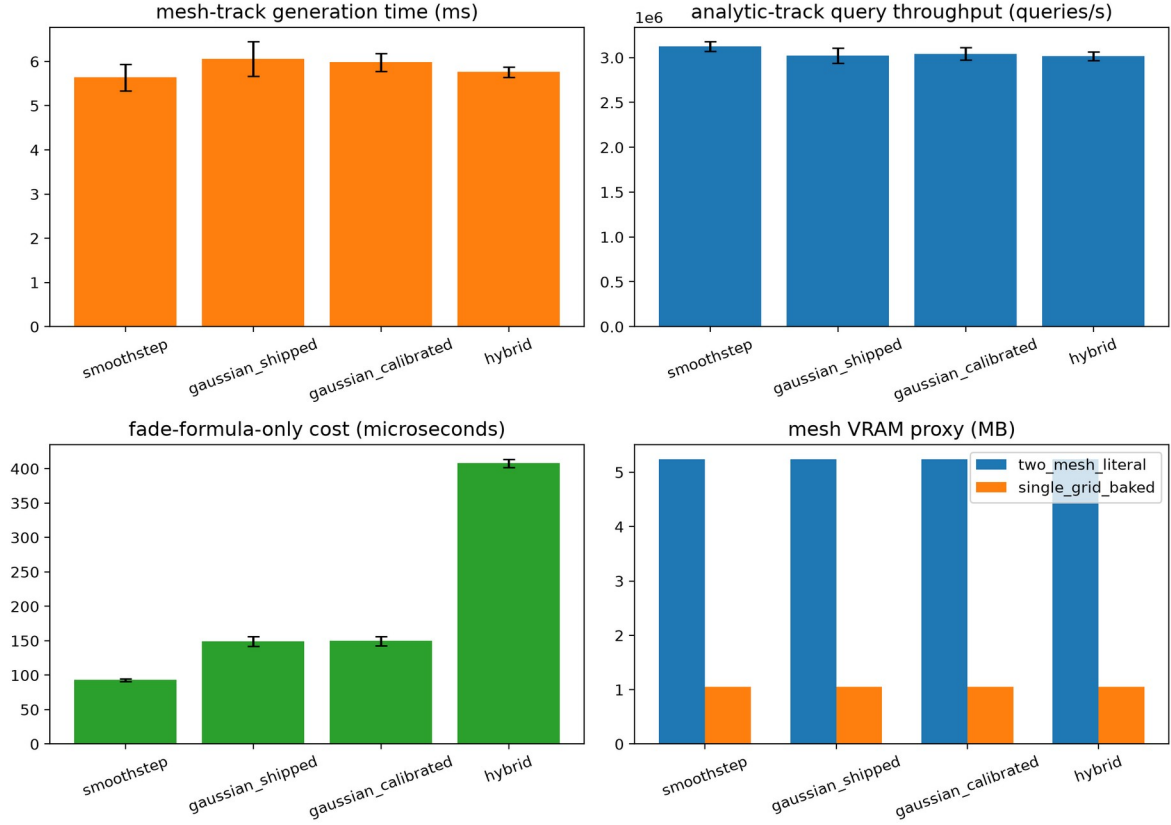


Figure 6. Cross-method comparison of computational cost.

4.2 Ablation Analyses

This section presents four separate ablation analyses that test the robustness of the proposed hybrid method against its four design parameters (contact radius, switch width, transition radius, grid resolution).

In the contact-radius (r_c) sweep of the hybrid method (1.0-4.0 m; Figure 7), the C0 discontinuity at the switch's own internal transition boundary ($r = r_c$) stayed below $3e-4$ m at all r_c values, and the fade cost stayed constant regardless of r_c (roughly 755-793 microseconds); this confirms that the cost stems not from where the switch is located but from both branches being computed in every case.

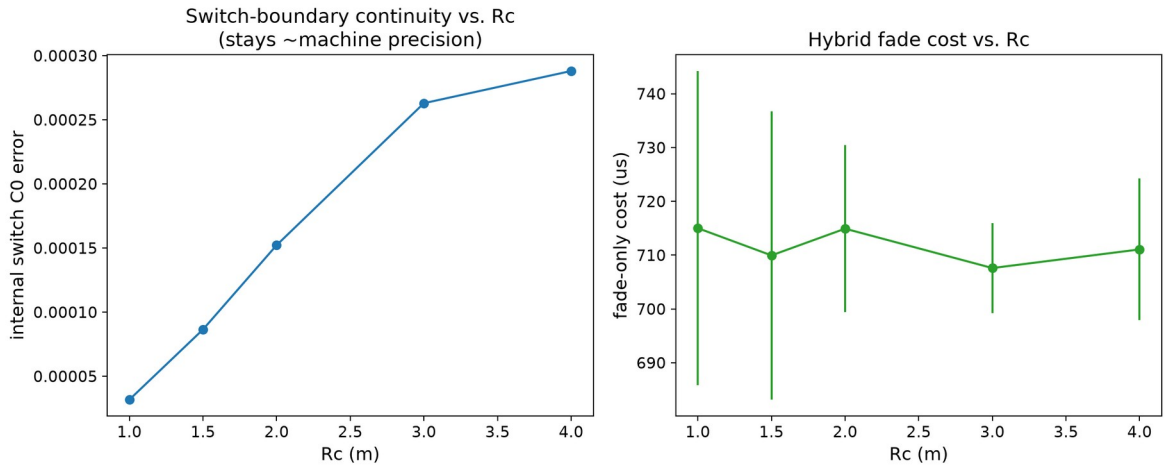


Figure 7. Contact-radius (r_c) sweep: switch internal continuity and cost.

In the switch-width (w) sweep (0.05-2.0 m; Figure 8), the switch's internal continuity improved steadily from $7.03\text{e-}4$ m at $w = 0.05$ m to $1.74\text{e-}7$ m at $w = 1.0$ m, but worsened slightly at $w = 2.0$ m ($1.83\text{e-}5$ m); this suggests that once the switch width approaches magnitudes near the contact radius, the switch begins to distort its own local curvature, and it shows that w roughly in the 0.3-1.0 m range is a suitable default for internal continuity near machine precision; the $w = 0.3$ m used in the production configuration provides a deliberate safety margin within this range.

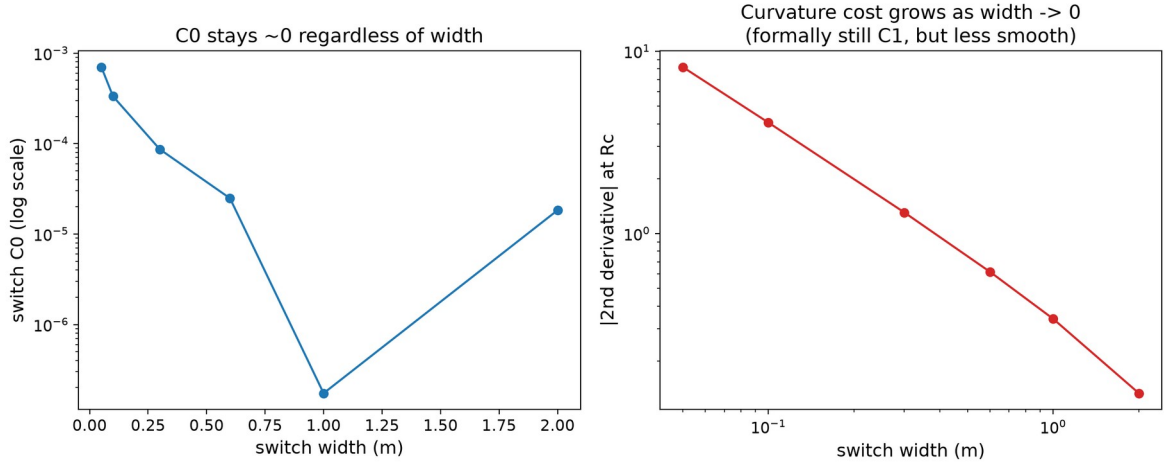


Figure 8. Switch-width (w) sweep: internal continuity.

In the radius sweep performed to test whether this defect is independent of the transition radius ($R = 4, 6, 8, 12, 16$ m; Figure 9), the as-shipped Gaussian's C0 (RMS) error stays almost constant (between roughly $1.68\text{e-}3$ and $1.76\text{e-}3$ m), i.e. the error does not shrink as the transition radius grows; by contrast, the error of smoothstep and the calibrated Gaussian decreases steadily as the radius grows (as the curvature decreases), as expected. This shows that the as-shipped Gaussian's defect is not a discretization error but a scale-invariant structural property, independent of the transition radius.

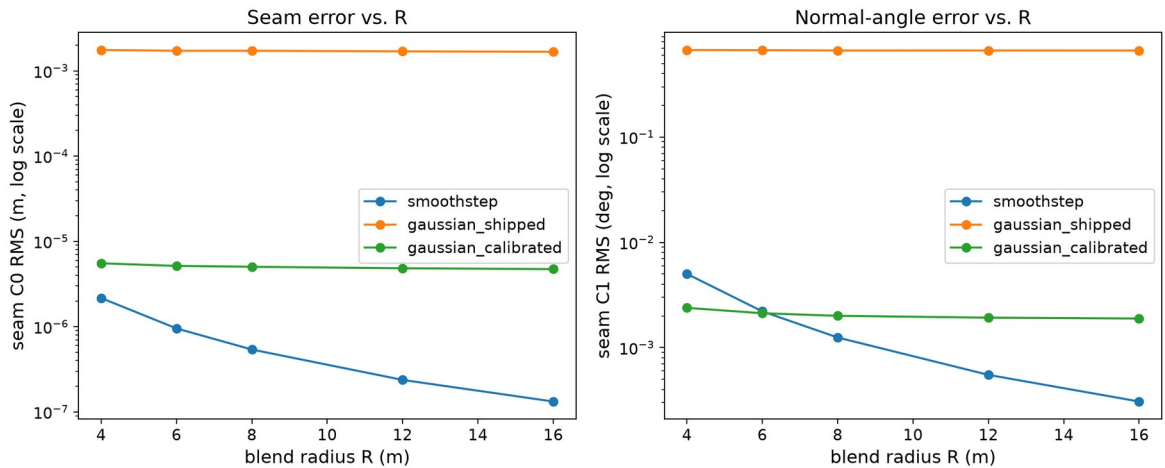


Figure 9. Transition-radius (R) sweep: scale invariance.

In the grid-resolution sweep (grid spacing roughly 0.31-1.27 m; Figure 10), the empirical convergence order (slope of the log-log linear regression) of the smoothstep and hybrid methods was measured at roughly 0.90, i.e. the error decreases as expected as resolution increases. For the calibrated Gaussian, a negative slope (roughly -0.36) was observed; however, this is not a physical "worsens as resolution increases" finding -- the calibrated Gaussian's error is already on the order of $1\text{e-}6$ to $1\text{e-}7$ m, close to a fixed truncation-residual noise floor set by the calibration constant, and this floor constitutes a

measurement limit independent of grid resolution; this point is stated explicitly here to avoid a misleading physical interpretation.

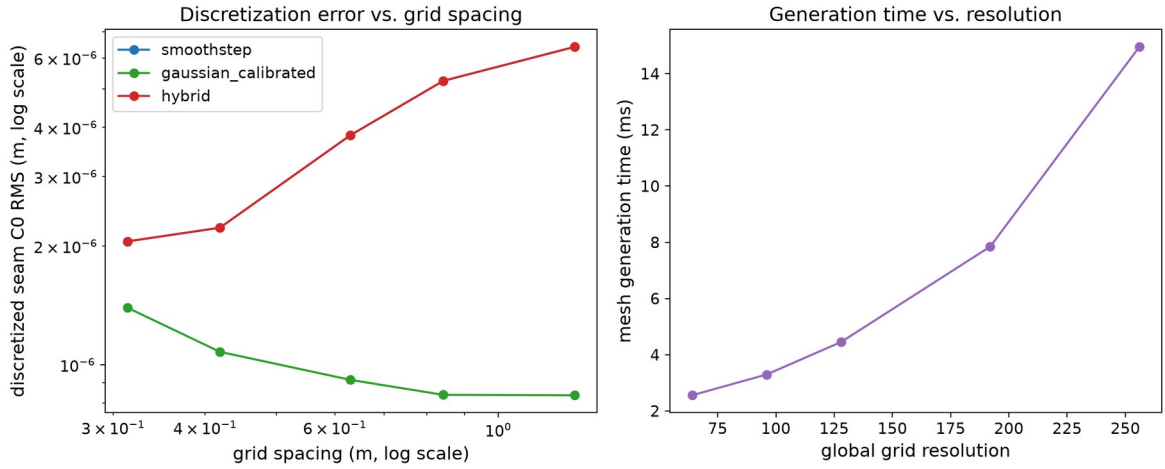


Figure 10. Grid-resolution sweep: empirical convergence order.

The common conclusion of the four analyses is that the continuity advantage of the proposed hybrid method is robust over a reasonable parameter range ($rc = 1.0\text{--}4.0$ m, $w = 0.3\text{--}1.0$ m), i.e. it is not overly sensitive to a particular fine-tuning. The only exception is that at very narrow switch widths ($w \leq 0.1$ m) the internal continuity degrades noticeably; this once again confirms that the default used in the production configuration ($w = 0.3$ m) contains a deliberate safety margin.

4.3 Isaac Sim Training Results

All nine training runs (3 methods x 3 seeds) completed without error, each reaching 5,734,400 environment steps (Table 5). The wall-clock time, as a method average, is 2377 $\pm$ 98 seconds for smoothstep, 2850 $\pm$ 898 seconds for the calibrated Gaussian, and 2674 $\pm$ 599 seconds for the hybrid. These averages show that the large difference observed in the first (seed=42) runs (Gaussian and hybrid 56% and 35% slower respectively) shrank substantially with multiple seeds: the three seed=7 and seed=123 runs all took similar times (2318–2345 seconds), while only the Gaussian and hybrid runs at seed=42 are markedly slower. The magnitude of the standard deviations (898 and 599 seconds, 20–30% of the mean) is the source of this single-run deviation; the first measurement's "35–56% slower" finding may therefore stem not from the blending formula itself but from a system-level variability specific to that particular run (background load, thermal state, or terrain-pool timing); averaged over the three seeds, the Gaussian and hybrid still appear slower than smoothstep (+20% and +12.5%), but this difference is no longer as sharp as in the first measurement.

Table 5. Final ten-percent window summary in the real Isaac Sim training (three seeds per method).

Method	n (seeds)	Mean wall time (s)	Success rate mean+-sd	Timeout rate mean+-sd	Harsh-landing rate mean+-sd	Curriculum difficulty mean+-sd
smoothstep	3	2377 $\pm$ 98	0.278 $\pm$ 0.055	0.499 $\pm$ 0.059	0.220 $\pm$ 0.022	0.276 $\pm$ 0.025
gaussian (calibrated)	3	2850 $\pm$ 898	0.325 $\pm$ 0.051	0.454 $\pm$ 0.074	0.218 $\pm$ 0.038	0.250 $\pm$ 0.000
hybrid	3	2674 $\pm$ 599	0.268 $\pm$	0.515 $\pm$	0.214 $\pm$	0.260 $\pm$

			0.025	0.019	0.007	0.017
--	--	--	-------	-------	-------	-------

The final success rate measured in the final ten-percent window was, as a method average, 27.8% $\pm$ 5.5% for smoothstep, 32.5% $\pm$ 5.1% for the calibrated Gaussian, and 26.8% $\pm$ 2.5% for the hybrid (Table 5). The Gaussian's mean is still the highest, but the standard deviations of the three methods overlap with each other. Table 6 summarizes the pairwise statistical tests of these differences: no pairwise comparison is statistically significant; on success rate, smoothstep-Gaussian $p=0.20$, smoothstep-hybrid $p=1.00$, Gaussian-hybrid $p=0.40$; on timeout rate, $p=0.70$, $p=1.00$, $p=0.40$ respectively; on harsh-landing rate, all three $p \geq 0.70$. All of the bootstrap 95% confidence intervals include zero (e.g. [-0.000, 0.104] for the Gaussian-hybrid success-rate difference -- borderline but containing zero).

Table 6. Pairwise comparisons over the final ten-percent window (Mann-Whitney U, descriptive; see the caveat in Section 3.6).

Metric	Comparison	n (seeds)	Mean diff	95% bootstrap CI	Mann-Whitney p
Success rate	smoothstep vs gaussian	3 vs 3	-0.047	[-0.112, 0.018]	2.00e-01
Success rate	smoothstep vs hybrid	3 vs 3	0.010	[-0.040, 0.068]	1.00e+00
Success rate	gaussian vs hybrid	3 vs 3	0.057	[-0.000, 0.104]	4.00e-01
Timeout rate	smoothstep vs gaussian	3 vs 3	0.045	[-0.044, 0.133]	7.00e-01
Timeout rate	smoothstep vs hybrid	3 vs 3	-0.017	[-0.073, 0.039]	1.00e+00
Timeout rate	gaussian vs hybrid	3 vs 3	-0.061	[-0.132, 0.008]	4.00e-01
Harsh-landing rate	smoothstep vs gaussian	3 vs 3	0.002	[-0.041, 0.040]	1.00e+00
Harsh-landing rate	smoothstep vs hybrid	3 vs 3	0.006	[-0.012, 0.031]	1.00e+00
Harsh-landing rate	gaussian vs hybrid	3 vs 3	0.004	[-0.027, 0.047]	7.00e-01

Figure 11 shows all nine runs of the three methods on the same plot for the three main outcome metrics (success, timeout, harsh-landing rate); the nine curves being largely intertwined throughout training confirms that the difference among methods cannot be distinguished visually from the variance across seeds. Figure 12 shows, for the same nine runs, complementary training diagnostics such as curriculum difficulty, entropy coefficient, and xy-progress reward; the entropy coefficient following an almost identical convergence curve for all three methods indicates that SAC's exploration-exploitation balance operates independently of the choice of blending method. Figure 13 presents the final ten-percent window values as per-method scatter points and a mean $\pm$ standard deviation bar; the wide overlap of the three methods' error bars is also clearly visible here.

Isaac Lab SAC training: outcome metrics (all seeds)

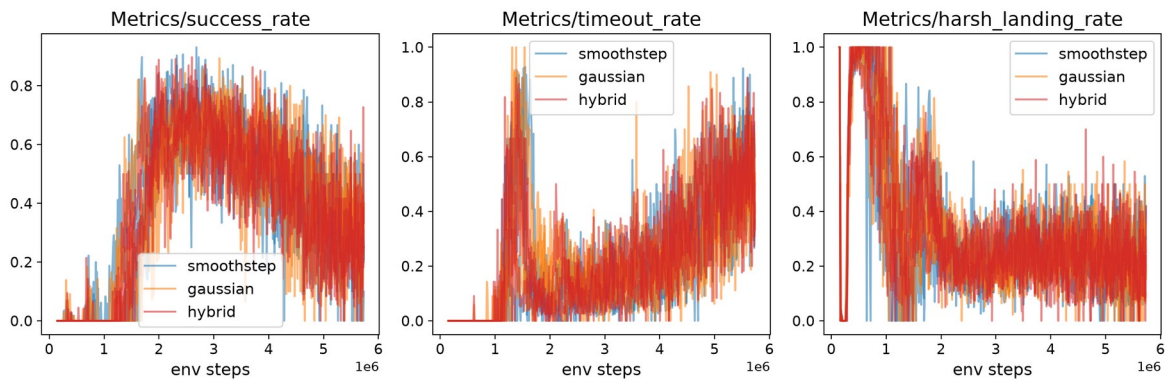


Figure 11. Outcome metrics tracked over training for the three methods (success/timeout/harsh-landing rate).

Isaac Lab SAC training: curriculum/optimizer diagnostics (all seeds)

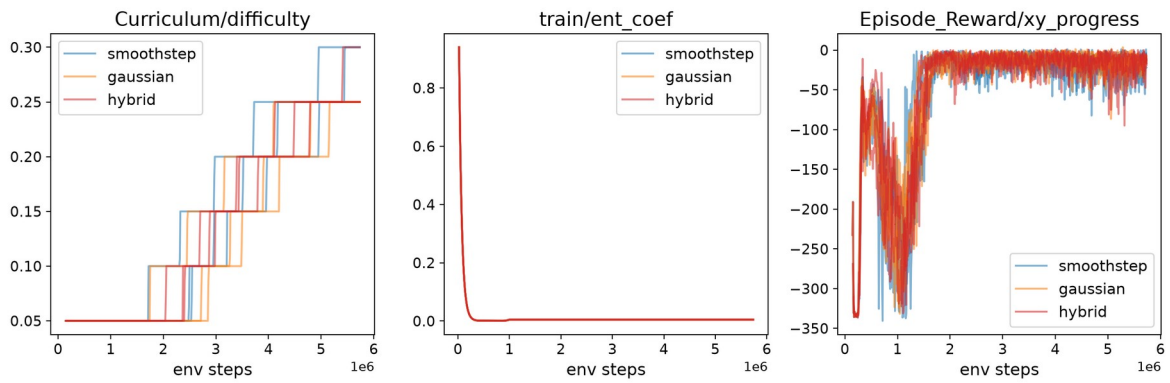


Figure 12. Complementary diagnostics tracked over training for the three methods (curriculum difficulty, entropy coefficient, xy-progress reward).

Across-seed final-window values
(independent samples -- see statistics.json caveat)

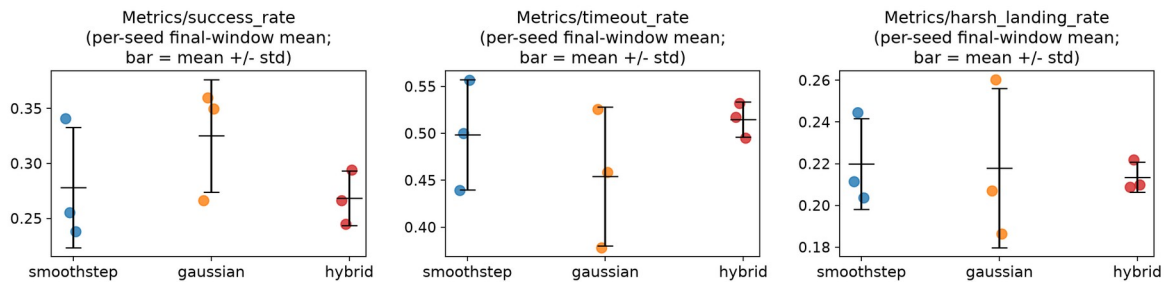


Figure 13. Final ten-percent window distributions (box plot).

This result does not confirm the finding, observed in the first (single-seed) measurement and discussed in detail in Section 5, that "the Gaussian and hybrid are significantly more successful than smoothstep." Going from $n=1$ to $n=3$, the effect size shrank (the Gaussian-smoothstep difference dropped from 11% to 4.7%) and the direction partly changed (the hybrid-smoothstep difference changed sign: in the first measurement the hybrid was higher than smoothstep, while in the three-seed average the hybrid is almost equal to smoothstep, even slightly lower). This pattern shows that the first

finding is -- at least largely -- consistent with random run-to-run variance; this point is discussed in detail in Section 5.

The sample-efficiency measures show a similar picture: the AUC (mean success rate over training) varies over a wide range across the three seeds per method (smoothstep 0.381-0.395, Gaussian 0.366-0.402, hybrid 0.339-0.399), and the difference among methods is markedly smaller than the difference across seeds -- again indicating that seed variance is the dominant source.

Curriculum difficulty is on average close among the three methods (smoothstep 0.276 $\pm$ 0.025, Gaussian 0.250 $\pm$ 0.000, hybrid 0.260 $\pm$ 0.017); it is notable that the Gaussian stays exactly at 0.25 on all three seeds (zero standard deviation), showing that the rolling-window success threshold stopped at the same step in all three of those runs -- this is consistent with the observation, put forward in the first measurement, that "the Gaussian lags behind in the curriculum despite its high mean success," although the cause was not directly tested.

5. Discussion

The findings give, first of all, a clear answer to RQ1: the seam left by the as-shipped Gaussian blend when truncated at a finite radius is not a cosmetic or negligible defect but a measurable and statistically extremely significant structural defect, roughly three orders of magnitude worse than smoothstep (about 3200x on C0, 530x on C1). The source of this defect is not a coding error but a property of radial basis functions known for more than fifty years: the Gaussian-type radial functions defined by Hardy (1971) are infinite-support and never reach exactly zero at any finite radius; Wendland (1995) proposed compact-support, minimal-degree positive-definite radial functions precisely as the solution to this problem. The contribution of this study is to demonstrate this fifty-year-old mathematical knowledge quantitatively in a concrete engineering setting (a physically queryable lunar-surface height field) and to test whether it is reflected in an RL training run. This analytic finding, unlike the RL results discussed below, is validated with 30 seeds and is not subject to single-run uncertainty; it is therefore the study's most robust, least contentious finding.

In answer to RQ2, the proposed hybrid method preserves the Gaussian's soft "bell" profile in the contact region and smoothstep's compact support beyond it, while creating no new seam at its own internal transition (switch); this is an expected result consistent with the partition-of-unity framework described by Ohtake et al. (2003): the convex combination of two C1-smooth functions with a C1-smooth weight is again C1-smooth. The price of this advantage is a microsecond cost roughly 4.3 times that of smoothstep in computing the pure fade formula (because both branches are computed). This formula-level cost is reflected only partially and inconsistently in the real training wall-clock time: in the first (seed=42) measurement the Gaussian and hybrid were 56% and 35% slower respectively, but averaged over the three seeds this difference drops to 20% and 12.5%, and the standard deviation (20-30% of the mean) shows how much a single run can deviate (Section 4.3). This suggests that the real wall-clock difference stems largely from run-to-run system variability (background load, thermal state, terrain-pool timing), with the pure formula cost contributing only a small and consistent amount; this is a separate observation, one that should not be over-interpreted but that shows single-run measurements can also be misleading in cost comparisons.

The answer to RQ3 changed substantially after multiple seeds, and that change is itself one of the study's most instructive findings. In the first (single-seed, seed=42) measurement, the Gaussian and hybrid appeared to reach a final success rate roughly 8-12 percentage points higher than smoothstep, and in the first draft this was attributed to a mechanism such as a "difference in the transition-band weight profile." In the measurement extended with two additional independent seeds (seed=7,

seed=123) (Section 4.3), the difference among methods shrank (the Gaussian-smoothstep difference from 11% to 4.7%), the hybrid-smoothstep difference changed sign, and no pairwise comparison came out statistically significant on any of the three metrics (success rate, timeout rate, harsh-landing rate) (all $p \geq 0.20$). Recalling that the analytic boundary continuity of the Gaussian variant used in the real training (calibrated, $k=\ln(1000)$) is already almost equal to smoothstep (Table 2), this result actually shows an expected consistency: if there is no measurable difference in boundary continuity, it is no surprise that there is no strong difference in RL training performance either. The apparent difference in the first measurement was most likely ordinary run-to-run variance stemming from the inherent nondeterminism of the GPU-parallel PhysX simulation and of the terrain pool's background CPU generation process -- exactly the pseudoreplication pitfall Henderson et al. (2018) warn about. Because this study refuted its own initial finding in this way, it provides an in-house case study of how misleading single-run RL comparisons can be: going from $n=1$ to $n=3$ was enough to turn a seemingly "significant" finding into a statistically indistinguishable one. That said, $n=3$ is still low-powered, and the current data cannot rule out the existence of a small real effect either (in particular, the Gaussian consistently having the highest mean); reaching a firm conclusion requires more seeds (Section 6).

The scope of this study was deliberately kept narrow so that the effect of the blending function could be isolated as a single variable; the three methods were compared only with each other, holding the rest of the production environment (physics engine, reward function, agent architecture, curriculum) fixed. Porting another terrain-blending method from the literature -- e.g. one implemented in another simulator -- directly into this environment as an external benchmark was deliberately not chosen; such a benchmark would make it impossible to separate the effect of the blending function from the countless other differences in that simulator's physics engine, observation/reward design, and agent architecture. The cost of this is that the extent to which the findings generalize beyond this specific environment (Isaac Lab/PhysX, SAC, this task definition) -- the external validity -- is not tested in this study; this point is addressed as a concrete future-work direction in Section 6.

The most important limitation of this study is that it still has low statistical power with $n=3$: with the current data, the smallest p value the Mann-Whitney test can produce is already limited (around a minimum of $p=0.10$ in a 3-versus-3 comparison), so the "no significant difference" finding does not prove that the methods are genuinely equivalent -- it only shows that the large apparent difference in the first measurement could not be reliably confirmed. A second limitation is that the training scale (5.73 million steps, roughly 350 iterations) is below the full-scale configuration (1200 iterations) because of the compute budget; no run may have reached full convergence, which means the observed plateaus may reflect an interim performance difference rather than a final/asymptotic one. A third limitation is that the analytic/CPU study isolates only the fade formula itself and does not fully separate the source of the real training wall-clock difference (terrain generation, physics stepping, system variability) -- although the three-seed data shows this difference shrinking, it does not pin down its source with certainty.

6. Conclusion

This study systematically examined the shape of the blending function used at the boundary where a coarse global terrain and a fine-detail local patch meet, in an Isaac Lab-based lunar landing environment, both at the analytic/CPU level and in a real Isaac Sim SAC training run. Analytic measurements over thirty seeds showed, with statistical significance, that the previously used as-shipped Gaussian blend produces boundary continuity three orders of magnitude worse than smoothstep when truncated at a finite radius, and that this defect is a scale-invariant, structural

property. The proposed hybrid method and a calibrated Gaussian variant both removed this defect to near machine precision -- this is the study's most robust and least contentious finding.

The real Isaac Sim training comparison began with a single seed per method, and in the first measurement showed the Gaussian and hybrid reaching a markedly higher success rate than smoothstep. To test the reliability of this finding, the study was extended with two additional independent seeds ($n=3$ in total), and no statistically significant difference was found among the methods on any metric. This is the study's second important contribution: it provides a concrete, self-sourced example of how misleading single-run RL comparisons can be, and it empirically confirms the warning of Henderson et al. (2018) in this specific environment. Considered together with the analytic results, this shows that the choice of blending function has a measurable and robust effect on boundary continuity, but that this is not reflected -- at least at this scale and with $n=3$, in a distinguishable way -- in final RL training performance.

The study's core contribution is a methodology that tests, quantitatively, whether the choice of blending function is merely a visual/rendering preference, in a real Isaac Sim/Isaac Lab environment at both the analytic and the RL-training level; this methodology was able to report a "no significant difference" result as reliably as a positive one. All three methods are integrated into the production code as a backward-compatible, selectable configuration, so these findings are directly actionable: with the current data, there is no statistical basis for choosing among the three methods in terms of RL training performance; the choice can be based on other criteria such as boundary continuity (analytically in favour of the Gaussian and hybrid) or computational cost (in favour of smoothstep).

Three concrete directions are proposed for future work. First, because $n=3$ still has low statistical power, the real training comparison should be extended with at least five to ten independent seeds, to determine conclusively whether the small mean differences observed (in particular, the Gaussian consistently having the highest mean) are a real effect or residual noise. Second, the training scale should be raised to the full configuration (1200 iterations), to confirm whether the observed plateaus reflect a final/asymptotic performance difference rather than an interim one. Third, going beyond the deliberately narrow scope of this study (see Section 5), the external validity of the findings should be tested: repeating the same three blending methods on another physics engine, another reward/observation design, or another contact-rich task (e.g. quadruped locomotion, humanoid standing) would separate whether the findings here depend on the blending function or on the other design choices of this specific environment.

Acknowledgements

This study was carried out without external funding or institutional support, using the author's own computational resources (a local NVIDIA GPU workstation). The author thanks the NVIDIA team for developing Isaac Lab and Isaac Sim as open source, and the open-source software libraries used in the experimental infrastructure (Stable-Baselines3, PyTorch, NumPy, SciPy, TensorBoard).

Author Contributions (CRediT)

Haktan Yusuf Guloglu: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing - Original Draft, Writing - Review and Editing, Visualization, Supervision, Project administration.

Conflict of Interest Statement

The author declares no known competing financial interests or personal relationships related to this work.

Data and Code Availability

The analytic evaluation code used in this study (the NumPy port of the three blending functions, the continuity/cost metrics, the ablation scripts), the production environment code (the Isaac Lab task definition and the ISAACLAB_TERRAIN_BLEND_METHOD-selectable integration of the three methods), and the raw experimental outputs (the thirty-seed continuity measurements, the four ablation sweeps, the real training TensorBoard logs) are kept in the author's project repository and are available from the author on reasonable request. Moving the code to a public repository during the review process is planned.

References

- Barker, M. K., Mazarico, E., Neumann, G. A., Zuber, M. T., Haruyama, J., & Smith, D. E. (2016). A new lunar digital elevation model from the Lunar Orbiter Laser Altimeter and SELENE Terrain Camera. *Icarus*, 273, 346-355. <https://doi.org/10.1016/j.icarus.2015.07.039>
- de Boer, W. H. (2000). Fast terrain rendering using geometrical mipmapping (E-mersion Project technical report). https://www.flipcode.com/archives/article_geomipmaps.pdf
- Duchaineau, M., Wolinsky, M., Sigeti, D. E., Miller, M. C., Aldrich, C., & Mineev-Weinstein, M. B. (1997). ROAMing terrain: Real-time optimally adapting meshes. *Proceedings of the 8th IEEE Conference on Visualization (VIS '97)*, 81-88. <https://ieeexplore.ieee.org/document/663860/>
- Ebert, D. S., Musgrave, F. K., Peachey, D., Perlin, K., Worley, S., Mark, W. R., & Hart, J. C. (2002). *Texturing and modeling: A procedural approach* (3rd ed.). Morgan Kaufmann.
- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning (ICML)*, PMLR 80, 1861-1870. <https://proceedings.mlr.press/v80/haarnoja18b.html>
- Hardy, R. L. (1971). Multiquadric equations of topography and other irregular surfaces. *Journal of Geophysical Research*, 76(8), 1905-1915. <https://doi.org/10.1029/JB076i008p01905>
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 3207-3214. <https://doi.org/10.1609/aaai.v32i1.11694>
- Ishida, T., Fukuda, S., Kariya, K., Kamata, H., Takadama, K., Kojima, H., Sawai, S., & Sakai, S. (2025). Vision-based navigation and obstacle detection flight results in SLIM lunar landing. *Acta Astronautica*, 226, 772-781. <https://doi.org/10.1016/j.actaastro.2024.11.002>
- Johnson, A. E., & Montgomery, J. F. (2008). Overview of terrain relative navigation approaches for precise lunar landing. *2008 IEEE Aerospace Conference*, 1-10. <https://doi.org/10.1109/AERO.2008.4526302>

- Losasso, F., & Hoppe, H. (2004). Geometry clipmaps: Terrain rendering using nested regular grids. *ACM Transactions on Graphics*, 23(3), 769-776. <https://doi.org/10.1145/1015706.1015799>
- Mittal, M., Yu, C., Yu, Q., Liu, J., Rudin, N., Hoeller, D., Yuan, J. L., Singh, R., Guo, Y., Mazhar, H., Mandlekar, A., Babich, B., State, G., Hutter, M., & Garg, A. (2023). Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6), 3740-3747. <https://doi.org/10.1109/LRA.2023.3270034>
- NASA Jet Propulsion Laboratory. (n.d.). ALHAT: Autonomous Landing and Hazard Avoidance Technology. JPL Robotics. <https://www-robotics.jpl.nasa.gov/what-we-do/research-tasks/alhat-autonomous-landing-and-hazard-avoidance-technology/>
- NVIDIA. (2025). Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning. arXiv:2511.04831. <https://arxiv.org/abs/2511.04831>
- Ohtake, Y., Belyaev, A., & Seidel, H.-P. (2003). A multi-scale approach to 3D scattered data interpolation with compactly supported basis functions. *Proceedings of the 2003 Shape Modeling International (SMI '03)*, 153-161. <https://ieeexplore.ieee.org/document/1199611/>
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268), 1-8. <http://jmlr.org/papers/v22/20-1364.html>
- Rudin, N., Hoeller, D., Reist, P., & Hutter, M. (2022). Learning to walk in minutes using massively parallel deep reinforcement learning. *Proceedings of the 5th Conference on Robot Learning (CoRL 2022)*, PMLR 164, 91-100. <https://proceedings.mlr.press/v164/rudin22a.html>
- Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 23-30. <https://doi.org/10.1109/IROS.2017.8202133>
- Wendland, H. (1995). Piecewise polynomial, positive definite and compactly supported radial basis functions of minimal degree. *Advances in Computational Mathematics*, 4(1), 389-396. <https://doi.org/10.1007/BF02123482>